

第二节：什么是 CPU

从 ISA 语义到单周期数据路径

CPU 的行为定义

CPU 每个时钟周期读取当前状态和当前指令，计算下一状态，并在时钟边沿更新 PC、寄存器或内存。

$$S_{t+1} = \textit{Execute}(S_t, \textit{Inst}_t)$$

状态项	含义
PC	下一条待取指令的地址
Reg[0..31]	32 个通用寄存器的值
DataMem[...]	数据内存的内容

ISA 规定什么

ISA 规定软件能观察到的 CPU 行为：有哪些寄存器、指令怎么编码、每条指令如何更新状态。

- 每条指令的语义决定硬件要读哪里、算什么、写哪里。

RV32I 寄存器

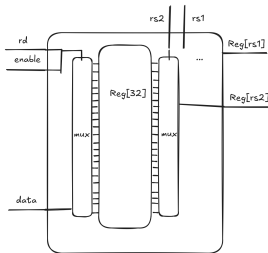
RV32I 有 32 个 32 位通用寄存器 x0 到 x31。

编号	ABI	常规用途	编号	ABI	常规用途
x0	zero	恒为 0	x16-x17	a6-a7	参数/返回值
x1	ra	返回地址	x18-x27	s2-s11	被调用者保存
x2	sp	栈指针	x28-x31	t3-t6	临时寄存器
x3	gp	全局指针	x5-x7	t0-t2	临时寄存器
x4	tp	线程指针	x8	s0/fp	保存值/帧指针
x9	s1	被调用者保存	x10-x17	a0-a7	参数/返回值

x0 读永远得到 0，写入被硬件直接丢弃。

寄存器堆端口

寄存器堆通常至少有两个读端口、一个写端口。



例子：add x5, x6, x7 从 `Reg[6]` 和 `Reg[7]` 读出操作数，结果写入 `Reg[5]`。

四类指令语义

类型	例子	状态更新
寄存器-寄存器运算	add x5,x6,x7	Reg[5] <- Reg[6] + Reg[7]; PC <- PC + 4
寄存器-立即数运算	addi x5,x6,12	Reg[5] <- Reg[6] + sext(12); PC <- PC + 4
内存读取	lw x5,8(x6)	Reg[5] <- DataMem[Reg[6]+8]; PC <- PC + 4
内存写入	sw x5,8(x6)	DataMem[Reg[6]+8] <- Reg[5]; PC <- PC + 4
条件分支	beq x5,x6,16	相等时 PC <- PC + 16, 否则 PC <- PC + 4

真实 32 位编码字段

硬件不会看到 add、lw 这样的文本，只会看到这些位段。

指令	32 位字段
add x5, x6, x7	0000000 00111 00110 000 00101 0110011
addi x5, x6, 12	000000001100 00110 000 00101 0010011
lw x5, 8(x6)	000000001000 00110 010 00101 0000011
sw x5, 8(x6)	0000000 00101 00110 010 01000 0100011
beq x5, x6, 16	0 000000 00110 00101 000 1000 0 1100011

从 add 开始

给定当前机器状态：

输入状态

PC = 0x80000000

instruction = add x5, x6, x7

Reg[6] = 20

Reg[7] = 22

这条指令的状态更新是：

语义

Reg[5] <- Reg[6] + Reg[7]

PC <- PC + 4

第一段：取指

一开始只有 PC，硬件先用 PC 从指令内存取出 32 位编码。



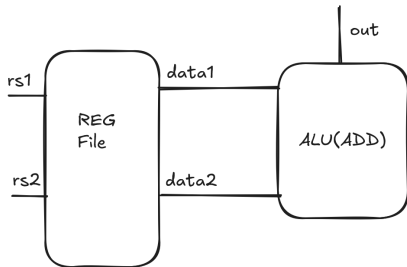
PC 是地址信号；指令内存根据这个地址给出当前指令编码。

第二段：解码字段

0000000 00111 00110 000 00101 0110011

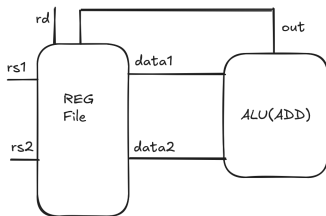
字段	值	对 add 的含义
rs2	7	第二个源寄存器 x7
rs1	6	第一个源寄存器 x6
rd	5	目的寄存器 x5
opcode/funct	R-type add	ALU 做加法

第三段：读寄存器与 ALU



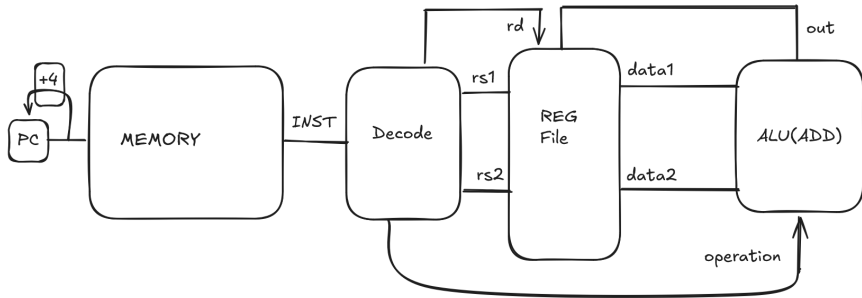
控制单元根据指令编码让 ALU 选择加法；ALU 输出只是组合逻辑结果，还没有改变状态。

第四段：写回



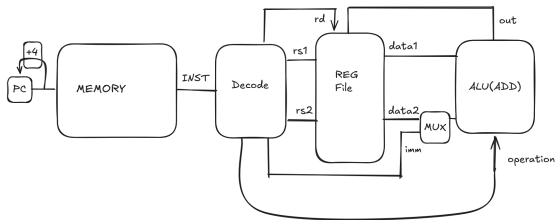
写回发生在周期边界；组合逻辑算出的结果只有通过寄存器堆写端口才会进入 CPU 状态。

add 的最小数据路径



addi: ALU 第二输入要选择

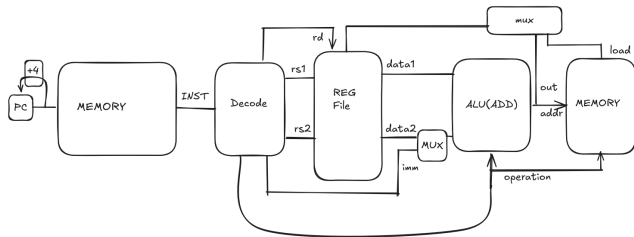
addi x5, x6, 12 的第一个操作数来自寄存器，第二个操作数来自立即数字段。



- `ALUSrc = 0`: 选择寄存器操作数。
- `ALUSrc = 1`: 选择立即数。

lw: 地址计算与内存读

lw x5, 8(x6) 先用 ALU 计算地址，再从数据内存读出数据，最后写回寄存器。



写回端还需要一个 MUX: 选择 ALU 结果或内存读出的数据。

sw: 写内存，不写寄存器

sw x5, 8(x6) 使用同一条地址计算路径，但状态更新落在内存里。

状态更新

Reg[6] + 8 = 0x108

Reg[5] = 0xAABBCCDD

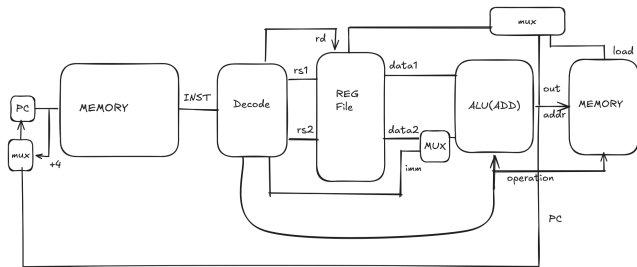
DataMem[0x108] <- 0xAABBCCDD

RegWrite = 0, MemWrite = 1

sw 不写任何通用寄存器。

beq: 选择下一条 PC

beq x5, x6, 16 读两个寄存器，用 ALU 做比较；结果为 0 时选择分支目标，否则顺序执行。



单周期实现的部件

部件	主要输入	主要输出	作用
PC 寄存器	PC_next、时钟	PC	保存取指地址
指令内存	PC	instruction[31:0]	给出当前指令
寄存器堆	rs1/rs2/rd、写回数据、写使能	read_data1/2	读源寄存器、写目的寄存器
控制单元	opcode/funct3/funct7	RegWrite、ALUSrc 等	生成控制信号
ALU	两个操作数、ALUControl	结果、Zero	运算、比较、地址计算
数据内存	地址、写数据、读写使能	读出数据	处理 lw/sw

同一套硬件每个周期只执行一条指令。

单周期设计的周期长度

单周期设计要求一条指令从取指到写回都在同一个时钟周期内完成。

指令	主要路径
----	------

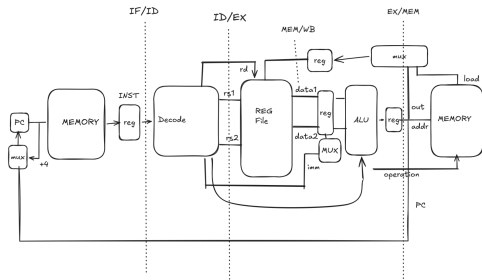
add	取指、读寄存器、ALU、写回
-----	----------------

lw	取指、读寄存器、算地址、访存、写回
----	-------------------

lw 的路径更长；所有指令却共用同一个周期，add 也必须按最慢的 lw 等。

为什么要流水线

把长路径拆成几段，在段间插入寄存器，周期长度就由某一段的延迟决定，而不是由整条取指到写回路径决定。



这不是改变 ISA 语义；同一条指令仍然完成同样的状态更新，只是多条指令可以在不同阶段重叠推进。